

Programming Principles I

Lecture 4

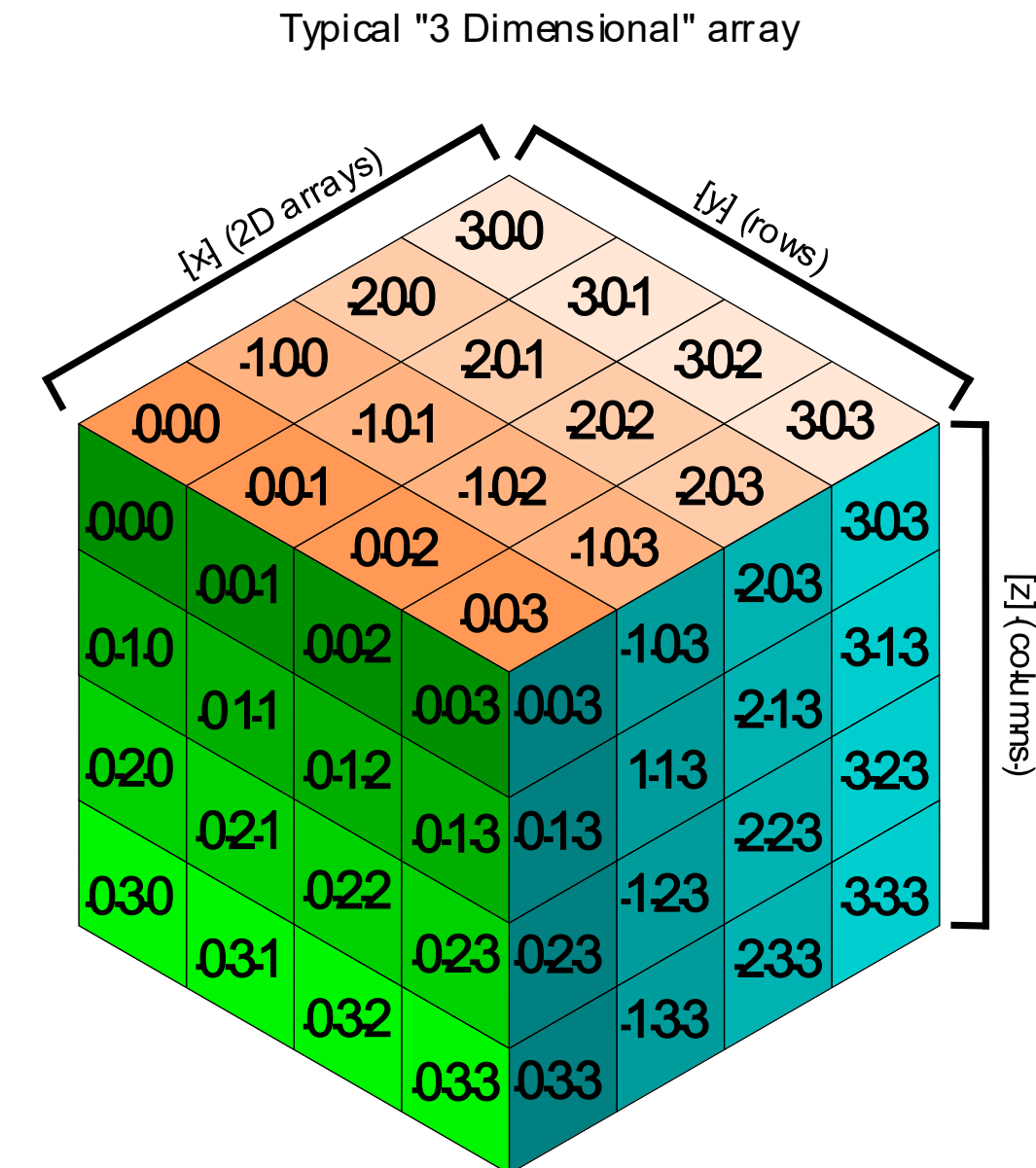


Zhiger Bolatov

Senior Lecturer

School of IT&I

- Arrays
- Types of Arrays
- Array declaration
- Accessing element of array
- Searching in Array
- 1D array samples
- String as array of chars
- Freopen



An **array** is a collection of elements, of the same type, stored in contiguous memory locations

Array of integers (int)

```
int arr[5] = {10, 20, 30, 40, 50};
```

Index:	0	1	2	3	4
	10	20	30	40	50
	1000	1004	1008	1012	1016

Contiguous memory locations

All elements are integers
(of type `int`).

Array of floating point numbers (double)

```
double arr[4] = {1.5, 2.0, 3.7, 4.1};
```

Index:	0	1	2	3
	1.5	2.0	3.7	4.1
	2000	2008	2016	2024

Contiguous memory locations

All elements are floating point
numbers (of type `double`).

Array of characters (char)

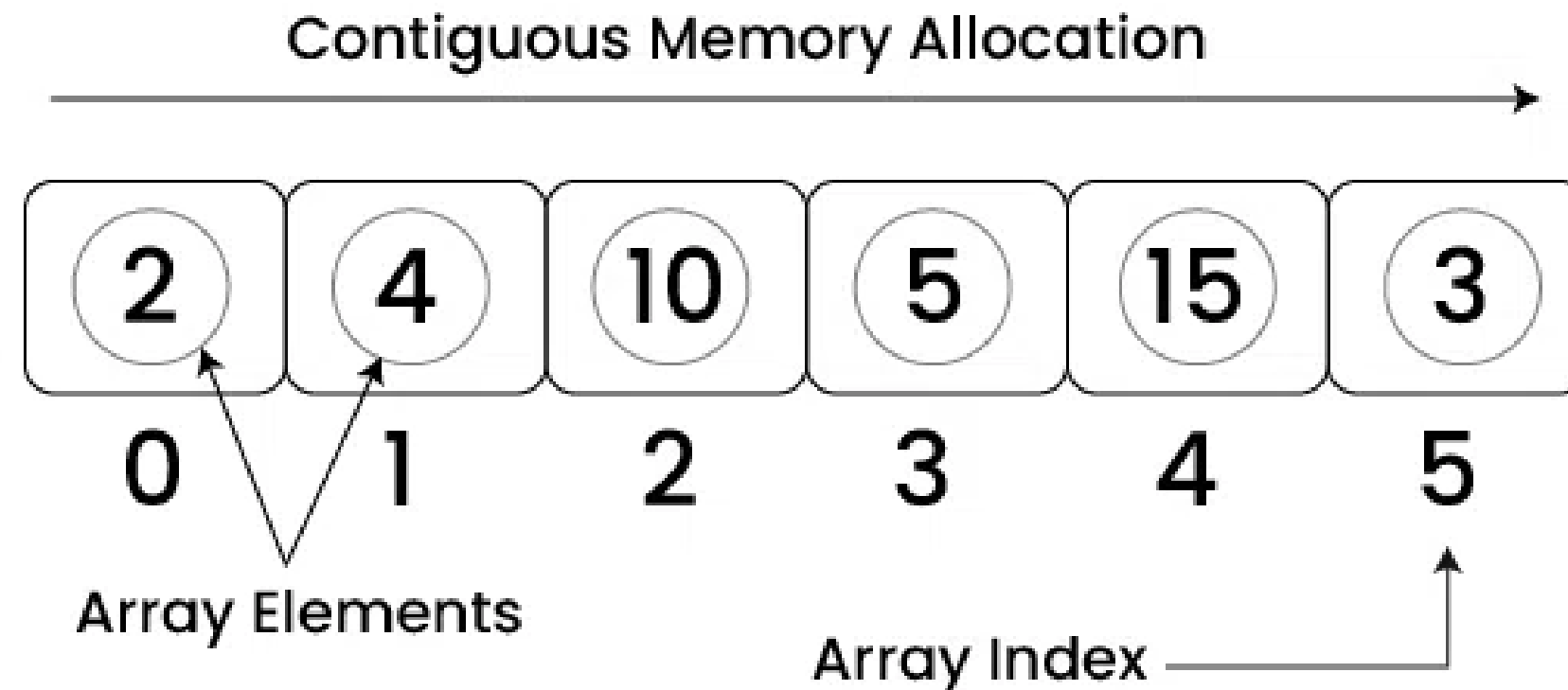
```
char arr[5] = {'A', 'B', 'C', 'D', 'E'};
```

Index:	0	1	2	3	4
	'A'	'B'	'C'	'D'	'E'
	3000	3001	3002	3003	3004

Contiguous memory locations

All elements are characters
(of type `char`).

An **array** is a collection of elements, of the same type, stored in contiguous memory locations



The array index indicates the position of the element (starting from 0)

Arrays are useful for storing lists of data like numbers, characters, or objects.

Advantages of Arrays

1 Fast access

Access any element directly using its index.

0	1	2	3	4
10	20	30	40	50

↑
`arr[3] = 40`

2 Easy to use

Store and manage many values with one variable.

```
int arr[5] = {10, 20, 30, 40, 50};
```

3 Contiguous memory

Elements are stored next to each other in memory.

10	20	30	40	50
1000	1004	1008	1012	1016

Contiguous memory locations

4 Efficient processing

Easy to process elements using loops.

```
for (int i = 0; i < 5; i++) {  
    cout << arr[i] << " ";  
}
```

5 Less code

Avoids creating many separate variables.

```
int a0 = 10;  
int a1 = 20;  
int a2 = 30;  
int a3 = 40;  
int a4 = 50;
```



```
int arr[5] =  
{10, 20, 30, 40, 50};
```

6 Memory efficient

Stores elements of the same type in an organized way.

int	int	int	int	int
10	20	30	40	50

Array Declaration

type name[**size**];

type

the type of
elements
(e.g., **int**, **double**, **char**)

name

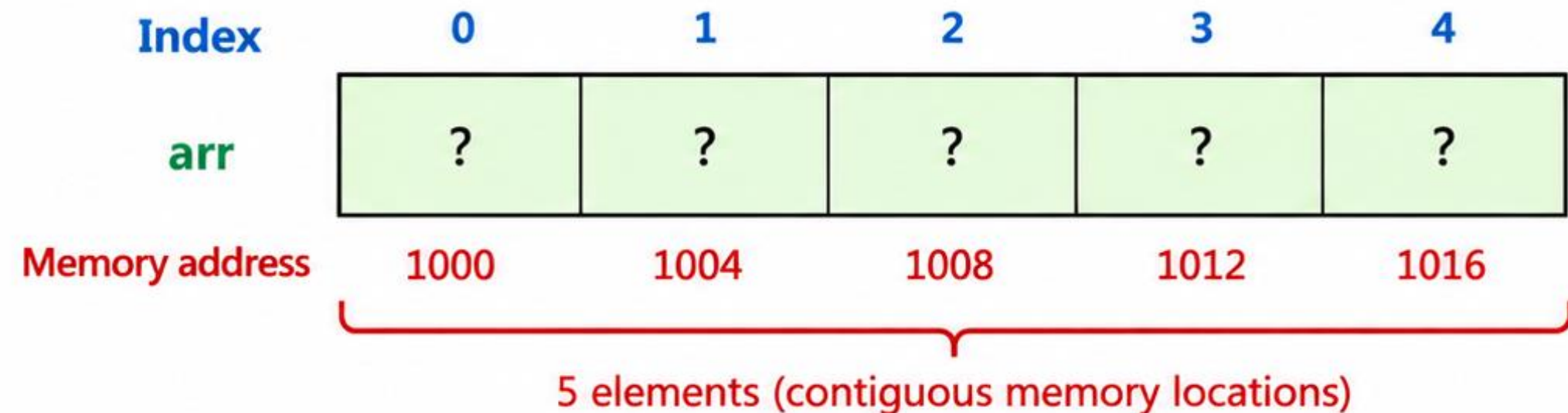
the array name
(e.g., **arr**)

size

the number of
elements
(e.g., 5)

Example

int arr[**5**];



Examples of Array Declaration

1 Array of integers

```
int numbers[5];
```

Declares an array of 5 integers.

2 Array of doubles

```
double values[10];
```

Declares an array of 10 double values.

3 Array of characters

```
char letters[26];
```

Declares an array of 26 characters.

4 Array of strings

```
string names[100];
```

Declares an array of 100 strings.

5 Array of booleans

```
bool flags[8];
```

Declares an array of 8 boolean values.

6 Array of floats

```
float measurements[3];
```

Declares an array of 3 float values.

Array Initialization

```
type name[size] = { value1, value2, ...};
```

type

the type of elements
(e.g., `int`, `double`, `char`)

name

the array name
(e.g., `arr`)

size

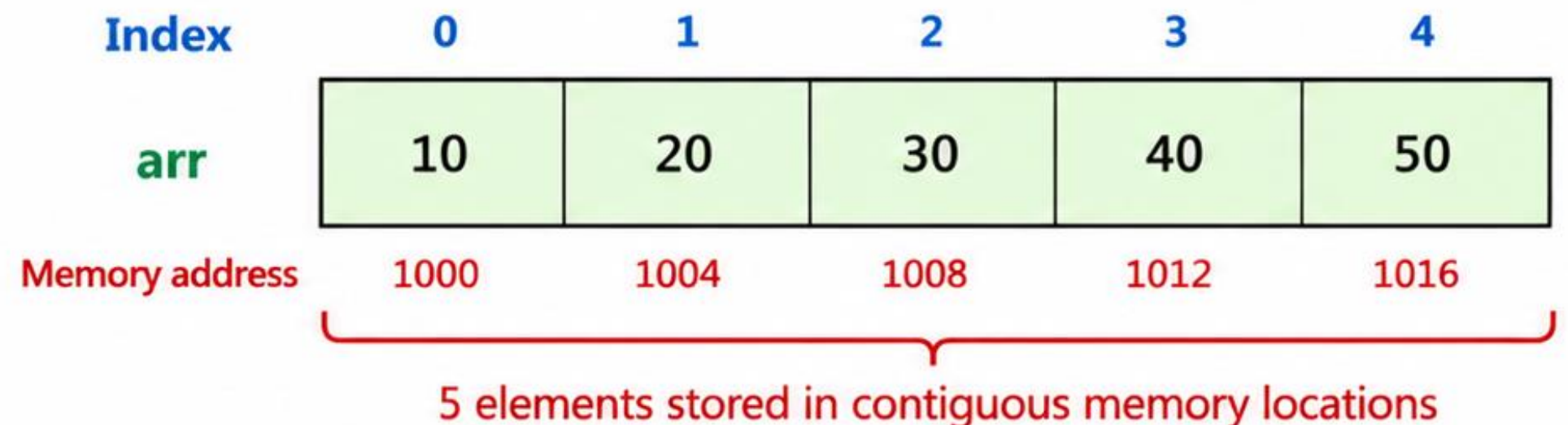
the number of
elements
(e.g., 5)

values

list of elements
of the same type

Example

```
int arr[5] = {10, 20, 30, 40, 50};
```



Examples of Array Initialization

1 Initialize with a list of values (integers)

```
int arr[5] = {10, 20, 30, 40, 50};
```

Index	0	1	2	3	4
arr	10	20	30	40	50

2 Partial initialization (integers)

```
int arr[5] = {1, 2, 3};
```

Index	0	1	2	3	4
arr	1	2	3	0	0

3 Initialize all elements with the same value

```
int arr[5] = {0};
```

Index	0	1	2	3	4
arr	0	0	0	0	0

4 Character array initialization

```
char letters[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Index	0	1	2	3	4	5
letters	'H'	'e'	'l'	'l'	'o'	'\0'

5 String array initialization

```
string names[5] = {"Alice", "Bob", "Charlie"};
```

Index	0	1	2	3	4
names	"Alice"	"Bob"	"Charlie"	""	""

6 Double array initialization

```
double values[4] = {1.5, 2.5, 3.5, 4.5};
```

Index	0	1	2	3
values	1.5	2.5	3.5	4.5

Ways to Define the Size of an Array in C++

1 Specify the size (no initial values)

```
int numbers[5];
```

- Creates an array of 5 integers.
- Elements are not initialized (they have indeterminate values).

Index	0	1	2	3	4
numbers	?	?	?	?	?

2 Specify the size and initial values

```
int numbers[5] = {10, 20, 30, 40, 50};
```

- Creates an array of 5 integers.
- Initializes each element with the given values.

Index	0	1	2	3	4
numbers	10	20	30	40	50

3 Omit the size (size is determined automatically)

```
int numbers[] = {10, 20, 30, 40, 50};
```

- Creates an array and initializes each element with the given values.
- The compiler determines the size from the number of initial values (size = 5).

Index	0	1	2	3	4
numbers	10	20	30	40	50

4 Works with different data types

The same array size rules apply to other data types (int, double, char, bool, string, etc.)

Integers

```
int a[] = {1, 2, 3};
```

Index	0	1	2
a	1	2	3

Doubles

```
double b[] = {1.5, 2.5};
```

Index	0	1
b	1.5	2.5

Characters

```
char c[] = {'A', 'B', 'C'};
```

Index	0	1	2
c	'A'	'B'	'C'

Booleans

```
bool flags[] = {true, false};
```

Index	0	1
flags	true	false

Strings

```
string names[] = {"Alice", "Bob"};
```

Index	0	1
names	"Alice"	"Bob"

5 Key Points

- You can specify the size explicitly.
- You can omit the size when initializing with values.
- The compiler determines the size from the number of initial values.
- Uninitialized elements have indeterminate values.
- These rules work with different data types (int, double, char, bool, string, etc.).
- You cannot declare an array without specifying the size or initial values.

6 Invalid example

```
int numbers[]; // Error!
```

✗ Error: the compiler cannot determine the array size because there are no initial values.

Accessing Array Elements

name[index]

name

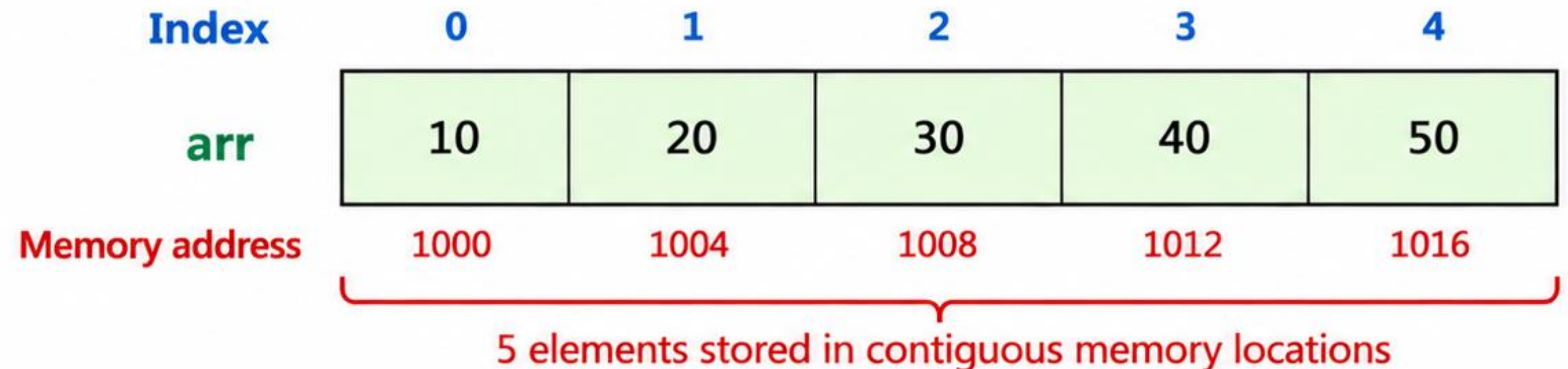
the array name
(e.g., **arr**)

index

the position of the element
(e.g., 0, 1, 2, ...)

Example

```
arr[0] // 10
arr[1] // 20
arr[2] // 30
arr[3] // 40
arr[4] // 50
```



Examples of Accessing Array Elements

1 Access a single element

```
int arr[5] = {10, 20, 30, 40, 50};  
cout << arr[2]; // outputs 30
```

Output:
30

Index	0	1	2	3	4
arr	10	20	30	40	50

↑
arr[2]

2 Access the first element

```
int arr[5] = {10, 20, 30, 40, 50};  
cout << arr[0]; // outputs 10
```

Output:
10

Index	0	1	2	3	4
arr	10	20	30	40	50

↑
arr[0]

3 Access the last element

```
int arr[5] = {10, 20, 30, 40, 50};  
cout << arr[4]; // outputs 50
```

Output:
50

Index	0	1	2	3	4
arr	10	20	30	40	50

↑
arr[4]

4 Use a variable to access an element

```
int arr[5] = {10, 20, 30, 40, 50};  
int i = 3;  
cout << arr[i]; // outputs 40
```

Output:
40

Index	0	1	2	3	4
arr	10	20	30	40	50

↑
arr[i]
(i = 3)

Example of Array Syntax

```
// declaration
```

```
int arr[5];
```

← declare an array of 5 integers

```
// initialization
```

```
arr[0] = 10;
```

```
arr[1] = 20;
```

```
arr[2] = 30;
```

```
arr[3] = 40;
```

```
arr[4] = 50;
```

← initialize the elements

```
// accessing elements
```

```
cout << arr[2]; // outputs 30
```

```
arr[4] = 100; // change element
```

After initialization:

Index	0	1	2	3	4
arr	10	20	30	40	50
Memory address	1000	1004	1008	1012	1016

5 elements in contiguous memory locations

After arr[4] = 100:

Index	0	1	2	3	4
arr	10	20	30	40	100
Memory address	1000	1004	1008	1012	1016

5 elements in contiguous memory locations

Simple Code Example: Array

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int arr[5] = {10, 20, 30, 40, 50};
6
7      for (int i = 0; i < 5; i++) {
8          cout << arr[i] << " ";
9      }
10     return 0;
11 }
```

Array of 5 integers initialized with values

Print all elements of the array

Array in memory

Index	0	1	2	3	4
arr	10	20	30	40	50
Memory address	1000	1004	1008	1012	1016

5 elements stored in contiguous memory locations

Output

10 20 30 40 50

1D array samples

Let's look at some sample use cases of 1D arrays:

```
int marks[5] = {85, 90, 78, 92, 88};  
for (int i = 0; i < 5; i++) {  
    cout << "Student " << i + 1 << ": " << marks[i] << endl;  
}
```

int marks[5]				
85	90	78	92	88
0	1	2	3	4

// output of the code

Student 1: 85

Student 2: 90

Student 3: 78

Student 4: 92

Student 5: 88

Use cases:

- Storing scores
- Managing to-do lists
- Tracking daily temperatures

String as array of chars

// **string** is actually a **character array**

```
char name[5] = {'A', 'l', 'i', 'c', 'e'};
```

- No null terminator (`\n`). Not a C-style string.
- Using string functions (e.g., `cout`) will lead to undefined behavior.



```
char name[] = "Alice";
```

- Automatically includes a null terminator (`\n`). This is a proper C-style string.
- Safe to use with C-style string functions.

Remember: The `\n` is essential for C-style string functions to know where the string ends!

Types of Arrays

One-Dimensional Array

1	2	3	3	4	5
---	---	---	---	---	---

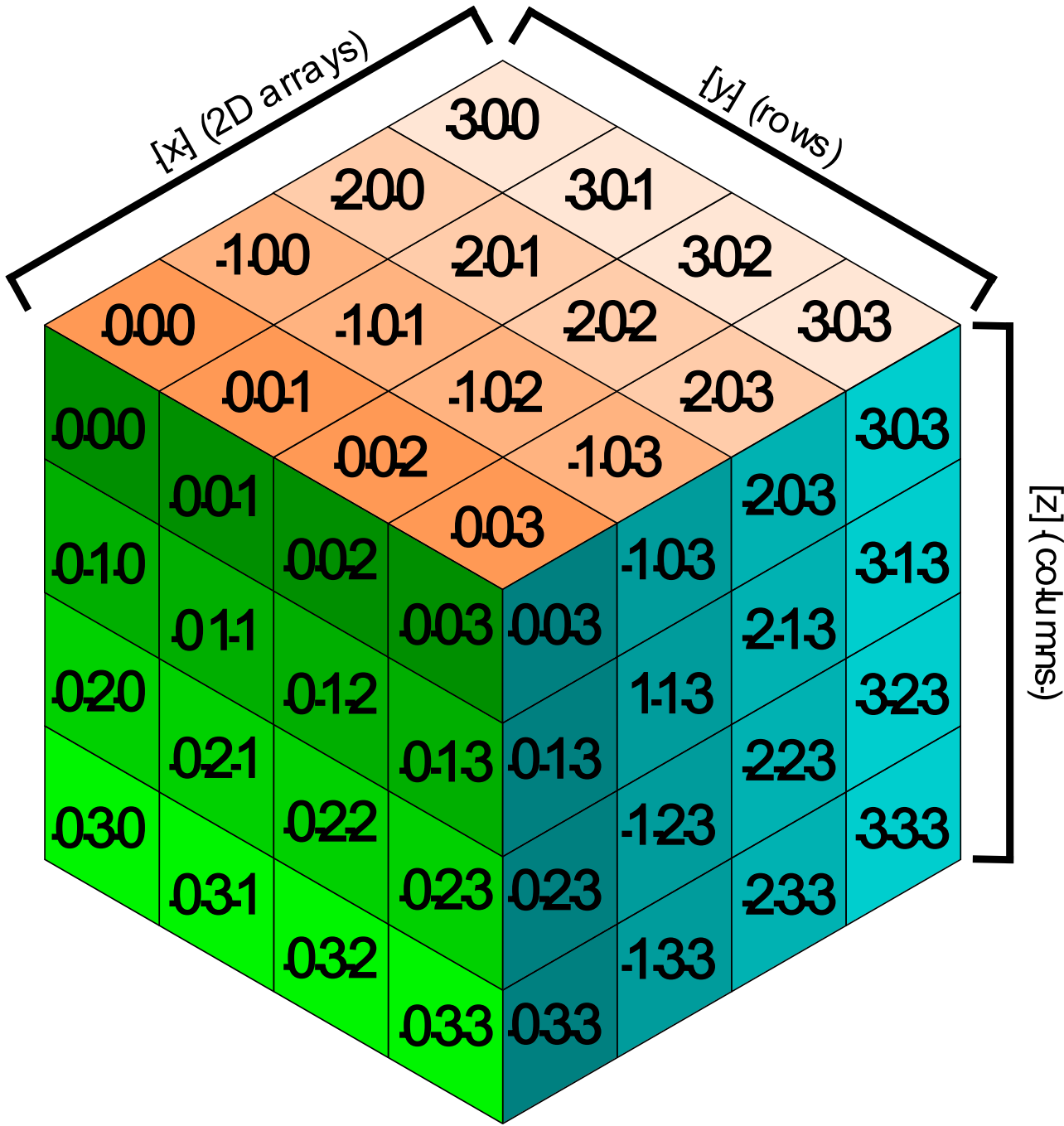
Two-Dimensional Array

1	2	3	6
4	5	6	

Multi-Dimensional Array

1	2	3	5	6	7
3	4	8	7	8	
			4	8	

Typical 3D array



Types of Arrays

Types of Arrays

1 One-dimensional array (1D)

A linear list of elements of the same type.

```
int arr[5] = {10, 20, 30, 40, 50};
```

Index	0	1	2	3	4
arr	10	20	30	40	50

2 Two-dimensional array (2D)

A table (matrix) of elements of the same type.

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

		Column index		
		0	1	2
Row index	0	1	2	3
	1	4	5	6

3 Character array

An array of characters, often used to store a string.

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Index	0	1	2	3	4	5
str	'H'	'e'	'l'	'l'	'o'	'\0'

4 Array of strings

An array where each element is a string (character array).

```
char names[3][10] = {
    "Ann",
    "Bob",
    "Kate"
};
```

Index	
0	"Ann"
1	"Bob"
2	"Kate"

Array declaration 2D

// initialization of two-dimensional array

```
dataType arrayName[rows][columns];
```

// 2D array with 2 rows and 3 columns

```
int numbers[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

```
int my4x4x4Array[4][4][4];
```

```
int value = 0;
```

```
for (int i = 0; i < 4; ++i) { // Loop for depth/layer
```

```
    for (int j = 0; j < 4; ++j) { // Loop for rows
```

```
        for (int k = 0; k < 4; ++k) { // Loop for columns
```

```
            my4x4x4Array[i][j][k] = value++;
```

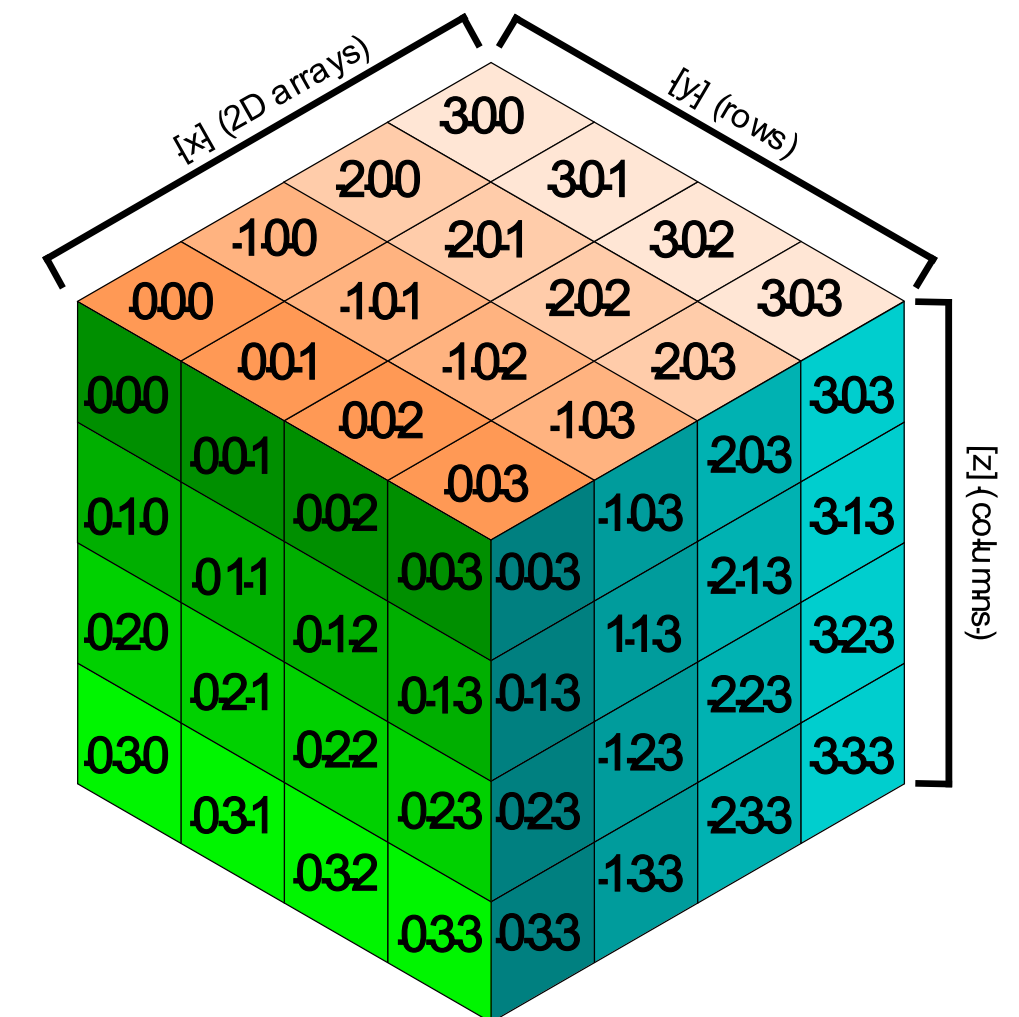
```
        }
```

```
    }
```

```
}
```

numbers

1	2	3
4	5	6



Accessing element of array

Each element in an array is accessed by its **index**, starting from 0.

numbers				
10	20	30	40	50
0	1	2	3	4

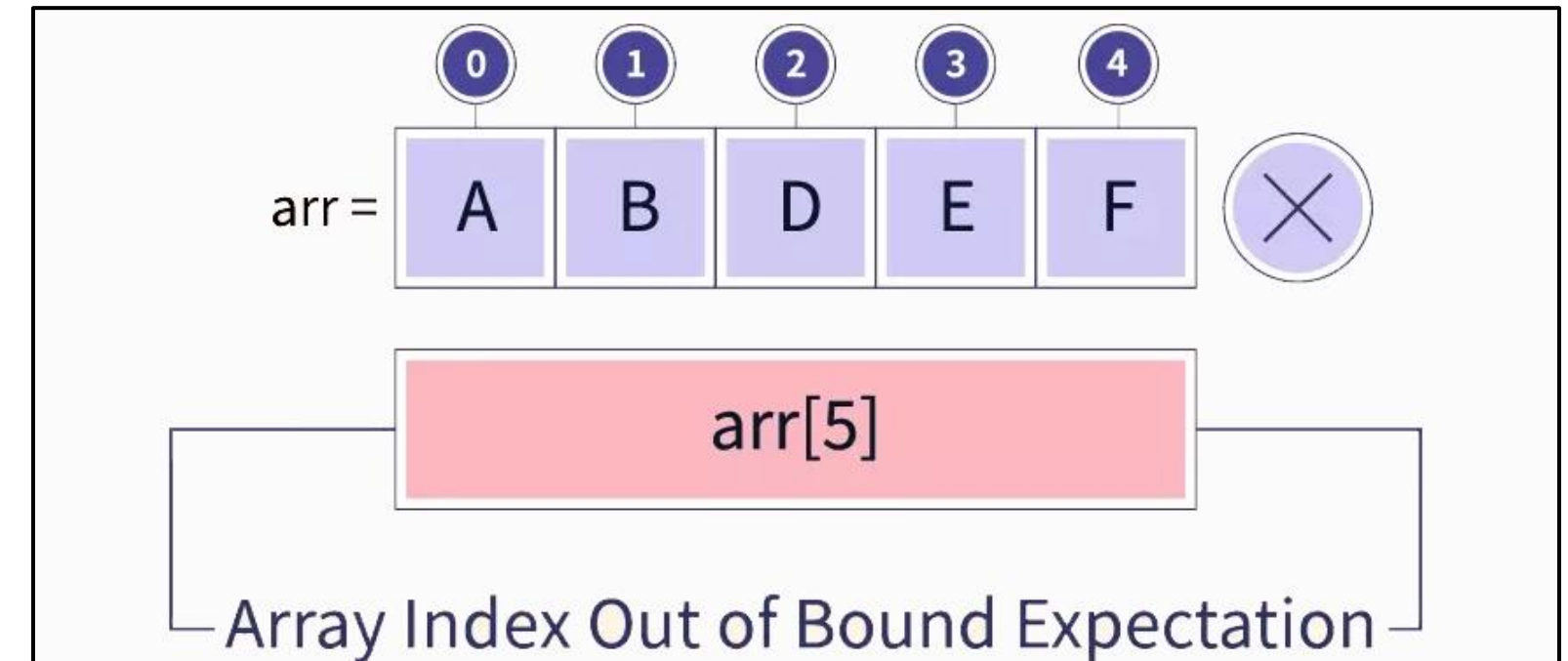
```
int numbers[5] = {10, 20, 30, 40, 50};  
cout << numbers[2]; // prints the third element (30)
```

Remember:

- numbers[0] is the first element.
- numbers[n-1] is the last, where n is the size of the array.

Array Out-of-Bounds

- In C++, arrays **do NOT** perform bounds checking at runtime.
- This means if you try to access an element using an index outside the declared size of the array, the compiler may not always give an error.
- However, this will lead to **Undefined Behavior**: your program might crash, produce incorrect results, or even create security vulnerabilities.
- Always be careful when working with array indices to ensure they are within the range from 0 to `array_size - 1`.



- **Access** — get an element by its index
- **Update** — change an element
- **Search** — find an element
- **Insert** — add an element at a specific position
- **Delete** — remove an element from a position
- **Minimum / Maximum** — find min and max
- **Count** — count elements satisfying a condition
- **Reverse** — reverse the order of elements
- **Sort** — arrange elements in ascending/descending order

Insertion of an Element into an Array (C++)

1 Idea

To insert a new element at a given position, we **shift all elements to the right** starting from the last element down to the position, and then place the new element at that position.

2 Example Code

```
#include <iostream>
using namespace std;

int main() {
    int arr[100] = {10, 20, 30, 40, 50}; // array capacity = 100
    int n = 5; // number of elements (currently used)
    int pos = 2; // position to insert (0 <= pos <= n)
    int val = 25; // value to insert

    // check if there is space and position is valid
    if (n == 100) {
        cout << "Array is full!" << endl;
        return 0;
    }
    if (pos < 0 || pos > n) {
        cout << "Invalid position!" << endl;
        return 0;
    }
    // shift elements to the right (from n-1 down to pos)
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    // insert the new element
    arr[pos] = val;
    n++; // increase the number of elements (not the array size)
    // print the first n elements
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

7 Key Points

- An array has a fixed capacity (size), e.g. arr[100] has 100 elements.
- We keep track of the number of elements using a variable n.
- To insert an element, shift elements to the right from the last element down to the position.
- After insertion, increase the number of elements (n++), not the array size.
- Check that the array is not full (n < 100) and the position is valid (0 <= pos <= n).

3 Initial Array

Array capacity = 100 (arr[100])
Number of elements (n) = 5

Index:	0	1	2	3	4	5	6	...	99
arr:	10	20	30	40	50			...	

5 elements are currently used (n = 5)

Total capacity = 100

4 Shift Elements to the Right

Index:	0	1	2	3	4	5	6	...	99
arr:	10	20	30	30	40	50		...	

Shift elements from the last element (index 4) down to index 2.

5 Insert the New Element

Index:	0	1	2	3	4	5	6	...	99
arr:	10	20	25	30	40	50		...	

Place **25** at **index 2**.

6 Result

Index:	0	1	2	3	4	5	6	...	99
arr:	10	20	25	30	40	50		...	

Number of elements is now n = 6

The new element is inserted and the number of elements increases by 1 (n = 6).

8 Important

- The position can be from 0 to n (inclusive). If pos == n, the element is added at the end.
- If n == 100, the array is full and you cannot insert a new element.
- Always shift elements from right to left to avoid overwriting values.
- Only the first n elements contain valid data. The remaining elements are unused.

Insertion When the Array is Full (Create a New Array)

1 Original array (full)

```
int arr[5] = {10, 20, 30, 40, 50};
```

Index	0	1	2	3	4
arr	10	20	30	40	50

The array is full (size = 5).
We cannot add another element to this array.

2 Create a new array with a larger size

```
int newArr[6]; // size 6 (5 + 1)
```

Index	0	1	2	3	4	5
newArr	?	?	?	?	?	?

We create a new array with one extra space for the new element.

3 Copy elements to the new array

```
for (int i = 0; i < 5; i++) {
    newArr[i] = arr[i];
}
```

Index	0	1	2	3	4	5
newArr	10	20	30	40	50	?

We copy all elements from arr to newArr.

4 Insert the new element

```
newArr[5] = 60; // insert new value
```

Index	0	1	2	3	4	5
newArr	10	20	30	40	50	60

Place the new element (60) at index 5.

5 Result

```
// newArr now contains 6 elements
for (int i = 0; i < 6; i++) {
    cout << newArr[i] << " ";
}
```

Index	0	1	2	3	4	5
newArr	10	20	30	40	50	60

The new array has 6 elements, and the element is successfully inserted.

6 Complete code

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int newArr[6];

    // copy elements
    for (int i = 0; i < 5; i++) {
        newArr[i] = arr[i];
    }

    // insert new element
    newArr[5] = 60;

    // print result
    for (int i = 0; i < 6; i++) {
        cout << newArr[i] << " ";
    }

    return 0;
}
```

Deletion of element from an array

To delete an element at a given position, we **shift the elements to the left** to fill the gap and then decrease the size by 1.

Example

Delete the element at **index 2** (value 30) from the array [10, 20, 30, 40, 50].

```
#include <iostream>
using namespace std;

int main() {
    int arr[100] = {10, 20, 30, 40, 50};
    int n = 5;          // current number of elements
    int pos = 2;        // index to delete
    // shift elements to the left
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    n--;                // decrease the size

    // print the array
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

1 Original array

Index	0	1	2	3	4
arr	10	20	30	40	50

We want to delete the element at **index 2** (value 30).

2 Shift elements to the left

Index	0	1	2	3	4
arr	10	20	40	50	

Shift elements from **index 3** and **4** to the left to fill the gap.

3 Decrease the size

Index	0	1	2	3
arr	10	20	40	50

After shifting, decrease the number of elements by 1 (**n--**).

4 Result

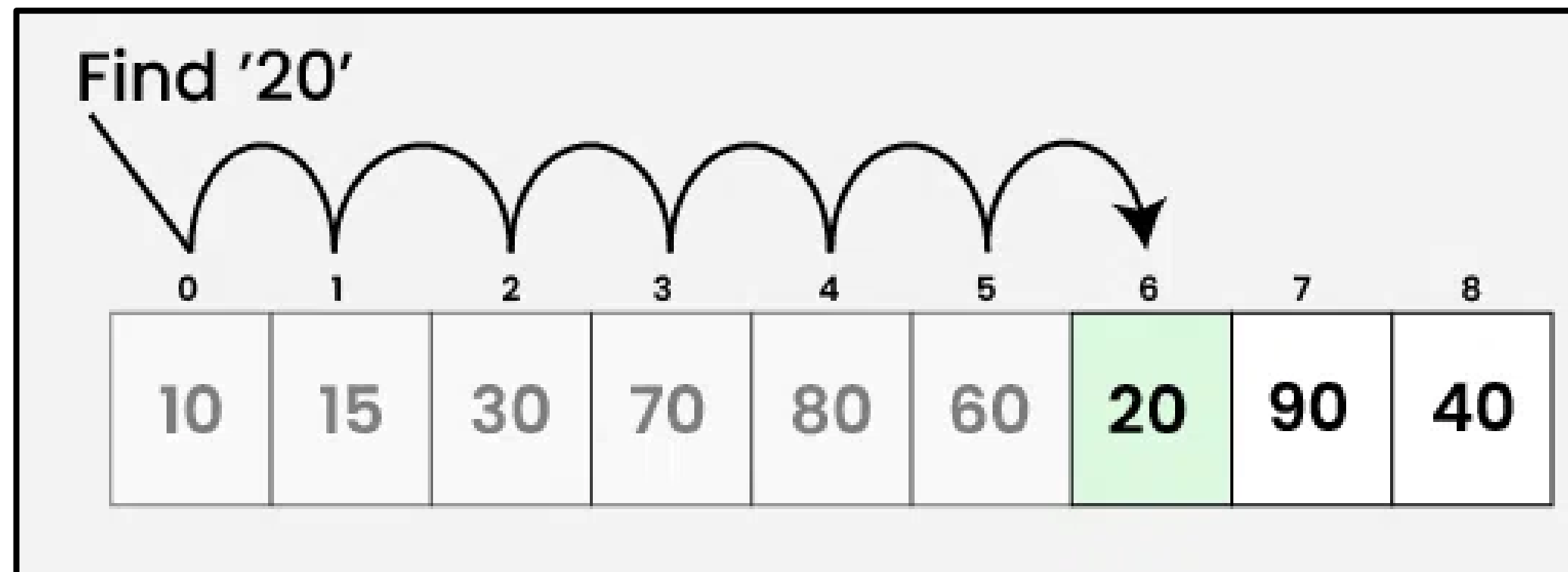
Index	0	1	2	3
arr	10	20	40	50

Element **30** is deleted. The new array is [10, 20, 40, 50] (**n = 4**).

Searching in Array

To find a specific value in an array, you typically use **linear search**

```
for (int i = 0; i < size; i++) {  
    if (arr[i] == target) {  
        // found at index i  
    }  
}
```



Search of element in an array (Linear Search)

Idea

To find a given element, we check each element of the array **one by one** from the first to the last until we find the element or reach the end.

Algorithm (Linear Search)

- 1 Start from index 0.
- 2 Compare each element with the target value.
- 3 If the element is found, return its index.
- 4 If we reach the end and do not find it, return **-1** (or print **"Not found"**).

C++ Example

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int target = 40;
    int index = -1;
    for (int i = 0; i < 5; i++) {
        if (arr[i] == target) {
            index = i;
            break; // stop at first occurrence
        }
    }
    if (index != -1)
        cout << "Found at index " << index;
    else
        cout << "Not found";
    return 0;
}
```

1 Initial array

Index	0	1	2	3	4
arr	10	20	30	40	50

Target value: **40**

2 Compare elements one by one

i = 0						i = 1						i = 2					
Index	0	1	2	3	4	Index	0	1	2	3	4	Index	0	1	2	3	4
arr	10	20	30	40	50	arr	10	20	30	40	50	arr	10	20	30	40	50
	↑							↑							↑		
	10 == 40 ?							20 == 40 ?							30 == 40 ?		
	No							No							No		

3 Element found

i = 3					
Index	0	1	2	3	4
arr	10	20	30	40	50
				↑	
				40 == 40 ?	
				Yes!	

Output:
Found at index 3

4 Element not found (example)

Target value: **100**

Index	0	1	2	3	4
arr	10	20	30	40	50

Output:
Not found

Binary Search (in a Sorted Array)

1 Idea

- Binary search **requires** the array to be **sorted** (in ascending order).
- At each step, we compare the target with the middle element.
- If they are equal → we found the element.
- If the target is smaller → discard the right part and search in the left part.
- If the target is larger → discard the left part and search in the right part.

2 Algorithm

- Set $left = 0$ and $right = n - 1$.
(Search range is $[left, right]$, both ends included)
- Find the middle index: $mid = (left + right) / 2$.
- If $arr[mid] == target$ → return mid .
- If $target < arr[mid]$ → discard the right part
($right = mid - 1$).
- If $target > arr[mid]$ → discard the left part
($left = mid + 1$).
- Repeat until $left > right$. If not found → return -1 .

3 C++ Implementation

```
int binarySearch(int arr[], int n, int target) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == target) {
            return mid; // element found
        }
        if (target < arr[mid]) {
            right = mid - 1; // search left part
        } else {
            left = mid + 1; // search right part
        }
    }
    return -1; // not found
}
```

4 Example 1: Search for 40 (element exists)

Array (sorted): [10, 20, 30, 40, 50, 60, 70] $n = 7$

Step 1

$left = 0$, $right = 6$
 $mid = (0 + 6) / 2 = 3$

Index:	0	1	2	3	4	5	6
arr:	10	20	30	40	50	60	70

Compare **40** with **40**.
 $arr[3] = 40$

Element found!
Return index 3.

6 Key Points

- Binary search works only on a **sorted array**.
- The search range is $[left, right]$ (both **ends included**).
- At each step we eliminate **half** of the remaining elements.
- Time complexity: $O(\log n)$.
- If the element is not found, the function returns **-1**.

5 Example 2: Search for 25 (element does not exist)

Array (sorted): [10, 20, 30, 40, 50, 60, 70] $n = 7$

Step 1

$left = 0$, $right = 6$
 $mid = (0 + 6) / 2 = 3$

Index:	0	1	2	3	4	5	6
arr:	10	20	30	40	50	60	70

Compare **25** with **40**.
 $25 < 40$ → search **left part**.
Set $right = mid - 1 = 2$.

Step 2

$left = 0$, $right = 2$
 $mid = (0 + 2) / 2 = 1$

Index:	0	1	2	3	4	5	6
arr:	10	20	30	40	50	60	70

Compare **25** with **20**.
 $25 > 20$ → search **right part**.
Set $left = mid + 1 = 2$.

Step 3

$left = 2$, $right = 2$
 $mid = (2 + 2) / 2 = 2$

Index:	0	1	2	3	4	5	6
arr:	10	20	30	40	50	60	70

Compare **25** with **30**.
 $25 < 30$ → search **left part**.
Set $right = mid - 1 = 1$.

Step 4

$left = 2$, $right = 1$
 $left > right$ → stop

Index:	0	1	2	3	4	5	6
arr:	10	20	30	40	50	60	70

Search range is empty.
Element is **not found**.
Return -1.

Finding the Smallest Value in an Array

Idea

- We go through the array and keep track of the smallest value.
- Start by assuming the first element is the smallest.
- Compare each element with the current smallest.
- If we find a smaller element, update it.
- After checking all elements, we get the smallest value.

Algorithm

- 1 Set min to the first element: $\text{min} = \text{arr}[0]$.
- 2 For each next element, compare $\text{arr}[i]$ with min.
- 3 If $\text{arr}[i] < \text{min}$, update $\text{min} = \text{arr}[i]$.
- 4 After the loop, min will be the smallest value.

C++ Example

```
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {40, 15, 27, 8, 35, 22};
    int n = 6;

    int min = arr[0]; // assume first is smallest
    for (int i = 1; i < n; i++) {
        if (arr[i] < min) {
            min = arr[i]; // update if smaller
        }
    }

    cout << "Smallest value: " << min;
    return 0;
}
```

Example: Find the smallest value

Array: [40, 15, 27, 8, 35, 22]

1 Initialization

Index	0	1	2	3	4	5
arr	40	15	27	8	35	22

↑
min = 40

Start with the first element.
min = 40

2 Compare next element (15)

Index	0	1	2	3	4	5
arr	40	15	27	8	35	22

↑
min = 15

$15 < 40$
Update **min = 15**

3 Compare next element (27)

Index	0	1	2	3	4	5
arr	40	15	27	8	35	22

↑
min = 15 (no change)

$27 < 15$?
No, keep **min = 15**

4 Compare next element (8)

Index	0	1	2	3	4	5
arr	40	15	27	8	35	22

↑
min = 8

$8 < 15$
Update **min = 8**

5 Continue (35, 22)

Index	0	1	2	3	4	5
arr	40	15	27	8	35	22

$35 < 8$? No
 $22 < 8$? No
Final **min = 8**

Compute Sum of All Values in an Array

Idea

- We go through the array and add each element to a variable that stores the sum.
- Start with **sum = 0**.
- For each element, add it to the sum.
- After the loop, the sum will be the total of all values.

Algorithm

- 1 Set **sum = 0**.
- 2 For each element in the array, add it to **sum**.
- 3 After the loop, **sum** contains the total value.

C++ Example

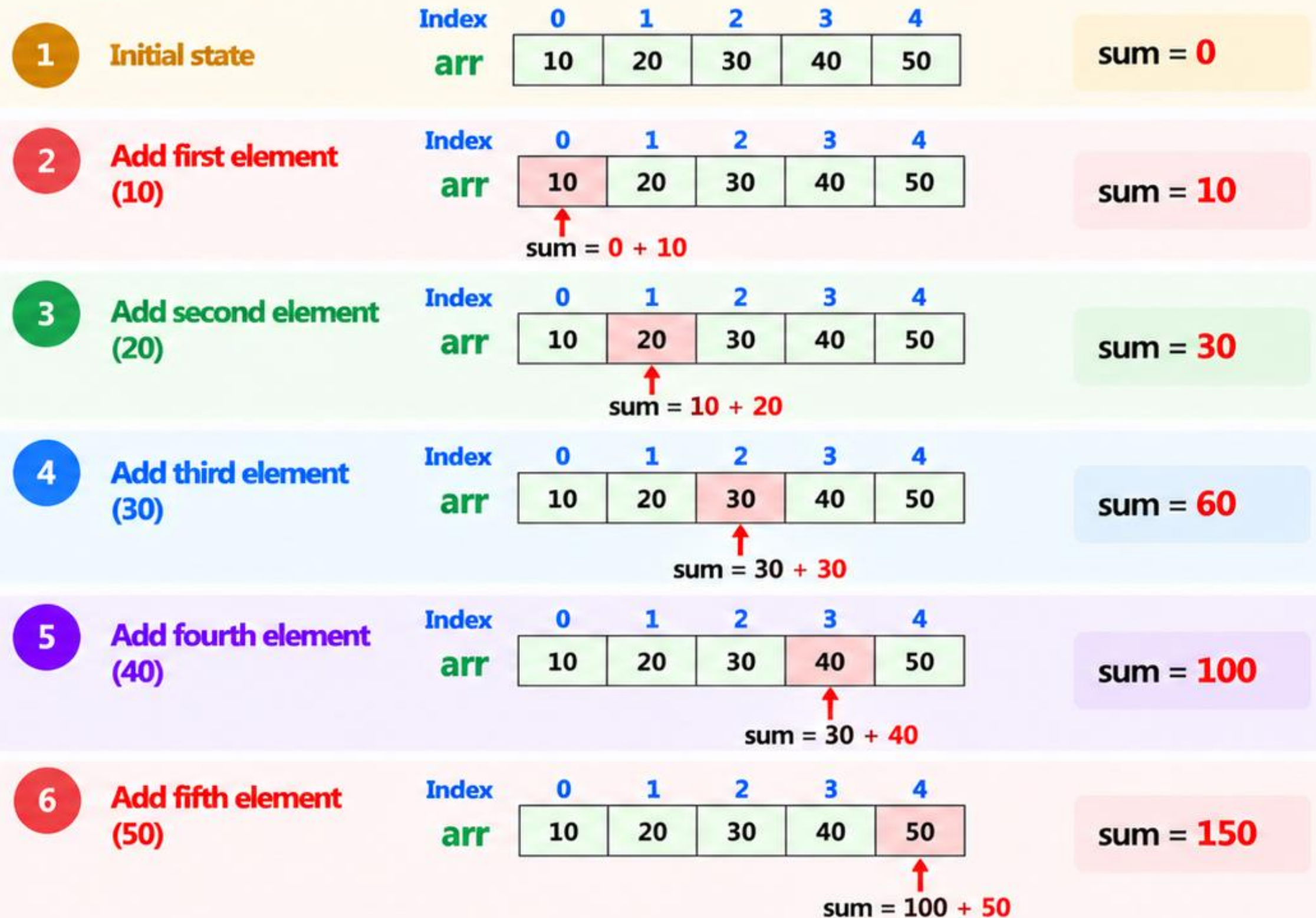
```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int n = 5;
    int sum = 0;           // initialize sum

    for (int i = 0; i < n; i++) {
        sum = sum + arr[i]; // add current element
    }

    cout << "Sum = " << sum;
    return 0;
}
```

Example: Compute the sum of [10, 20, 30, 40, 50]



Adding Two Arrays

Idea

- We have two arrays of the same size.
- We add corresponding elements at the same index.
- The result is a new array where **$\text{result}[i] = a[i] + b[i]$**

Algorithm

- 1 Read the two arrays a and b (of size n).
- 2 For each index i from 0 to n - 1, compute **$\text{result}[i] = a[i] + b[i]$** .
- 3 The result array contains the sum of the corresponding elements.

C++ Example

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int a[5] = {1, 2, 3, 4, 5};
    int b[5] = {10, 20, 30, 40, 50};
    int result[5];

    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i];
    }
    for (int i = 0; i < n; i++) {
        cout << result[i] << " ";
    }
    return 0;
}
```

Example: Add two arrays

Array a

Index	0	1	2	3	4
a	1	2	3	4	5

Array b

Index	0	1	2	3	4
b	10	20	30	40	50

Step-by-step addition

1 Index 0

$a[0] = 1$
 $b[0] = 10$

+

$1 + 10 = 11$

↓

result[0]

11

2 Index 1

$a[1] = 2$
 $b[1] = 20$

+

$2 + 20 = 22$

↓

result[1]

22

3 Index 2

$a[2] = 3$
 $b[2] = 30$

+

$3 + 30 = 33$

↓

result[2]

33

4 Index 3

$a[3] = 4$
 $b[3] = 40$

+

$4 + 40 = 44$

↓

result[3]

44

5 Index 4

$a[4] = 5$
 $b[4] = 50$

+

$5 + 50 = 55$

↓

result[4]

55

Result array

Index	0	1	2	3	4
result	11	22	33	44	55

Sorting an Array

1 What is Sorting?

Sorting an array is the process of arranging the elements in a specific order, usually in **ascending order** or **descending order**.

Unsorted array

	0	1	2	3	4
arr	40	10	50	20	30

2 Result of Sorting

Ascending order

Elements are arranged from smallest to largest.

	0	1	2	3	4
arr	10	20	30	40	50

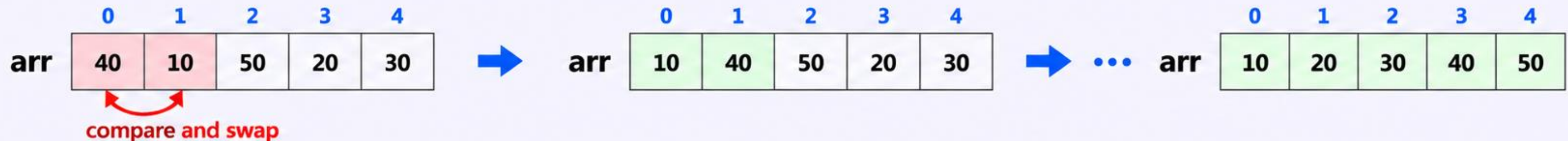
Descending order

Elements are arranged from largest to smallest.

	0	1	2	3	4
arr	50	40	30	20	10

3 Main Idea of Sorting

Sorting algorithms repeatedly **compare** elements and **swap** them if they are in the wrong order.



- The algorithm goes through the array multiple times.
- At each step, it compares elements and, if needed, swaps them to move them to the correct position.
- After all steps, the array is arranged in the desired order.

Bubble Sort

1 Idea

- Repeatedly compare adjacent elements and swap them if they are in the wrong order.
- After each pass, the largest element "bubbles up" to the end of the array.
- Repeat until the array is sorted.

2 C++ Code

```
#include <iostream>
using namespace std;

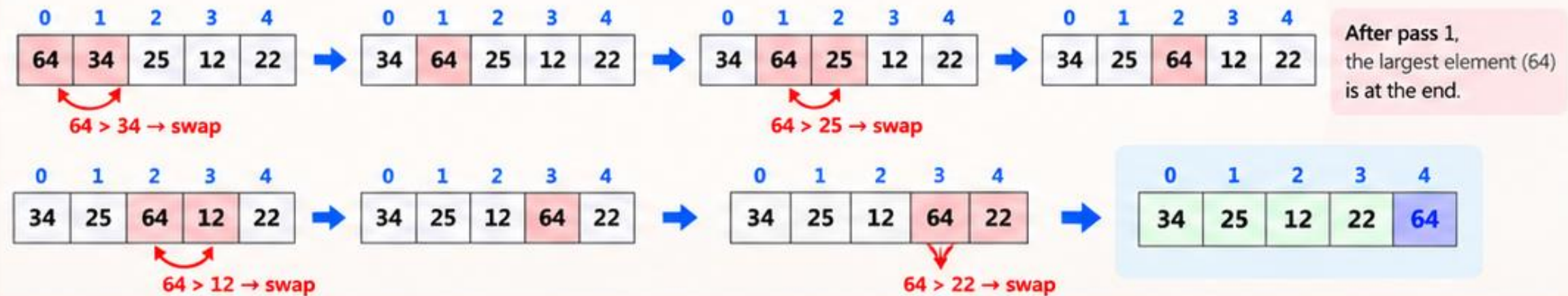
int main() {
    int arr[] = {64, 34, 25, 12, 22};
    int n = 5;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    // print result
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

3 Step-by-Step Example

Initial array: [64, 34, 25, 12, 22]

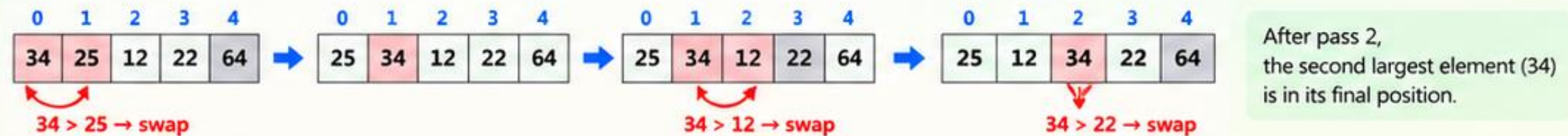
Pass 1

Compare adjacent elements from left to right.



Pass 2

Repeat for the remaining elements (0 to 3).



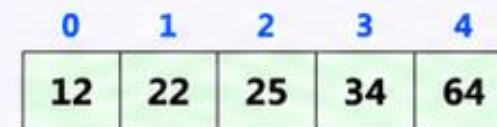
Pass 3

Repeat for the remaining elements (0 to 2).



Pass 4

Repeat for elements (0 to 1).



The array is sorted.
No more passes needed.

4 Result

Sorted array: [12, 22, 25, 34, 64]

5 Key Points

- Compare adjacent elements and **swap** if they are in the wrong order.
- After each pass, the largest remaining element moves to the end.
- The algorithm makes $(n - 1)$ passes.
- Time complexity: $O(n^2)$ (simple but not efficient for large arrays).

Selection Sort

1 Idea

- Find the **smallest element** in the unsorted part of the array.
- Swap it with the first element of the unsorted part.
- Repeat for the remaining elements until the array is sorted.

2 C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = 5;

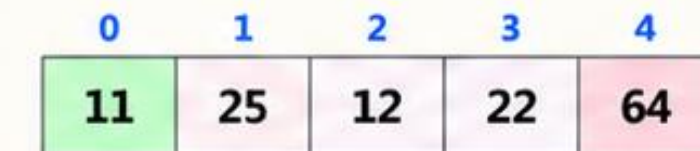
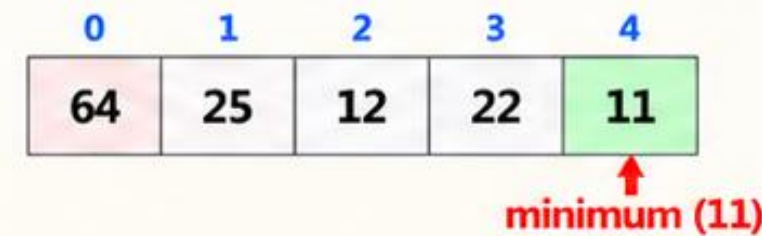
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // swap the minimum element
        // with the first element
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    // print result
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

3 Step-by-Step Example

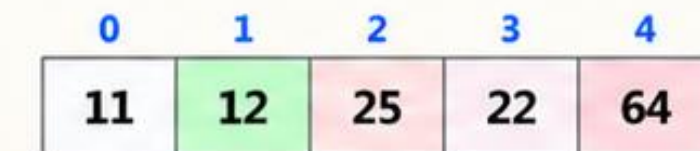
Step 1

Find the smallest element in the whole array and swap it with the first element.



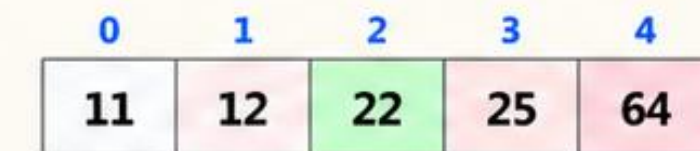
Step 2

Find the smallest element in the remaining part and swap it with element at index 1.



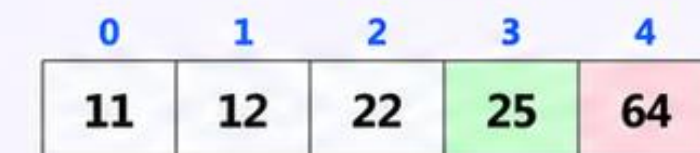
Step 3

Find the smallest element in the remaining part and swap it with element at index 2.



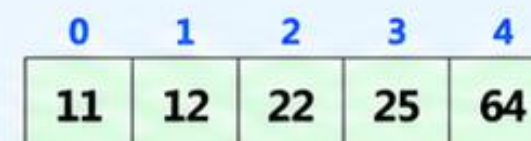
Step 4

Find the smallest element in the remaining part and swap it with element at index 3.



4 Result

Sorted array (ascending order):



5 Key Points

- Repeatedly find the minimum element in the unsorted part.
- Swap it with the first element of the unsorted part.
- After each step, the left part becomes sorted.
- Time complexity: $O(n^2)$ (simple but not efficient for large arrays).
- Sorts in ascending order in this example.

Insertion Sort

1 Idea

- Build a **sorted part** on the left side of the array.
- Take the next element and **insert** it into the correct position in the sorted part.
- Repeat for all elements.

2 C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 2, 4, 6, 1};
    int n = 5;

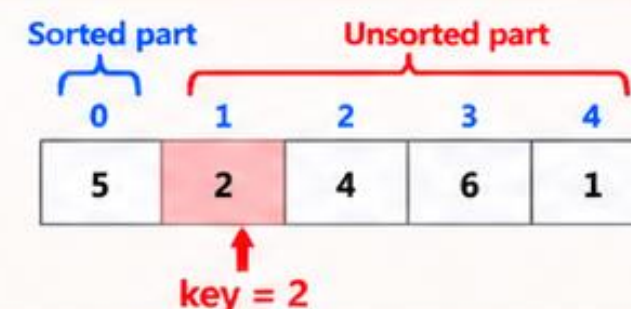
    for (int i = 1; i < n; i++) {
        int key = arr[i]; // element to insert
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // shift right
            j--;
        }
        arr[j + 1] = key; // insert element
    }

    // print result
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

3 Step-by-Step Example

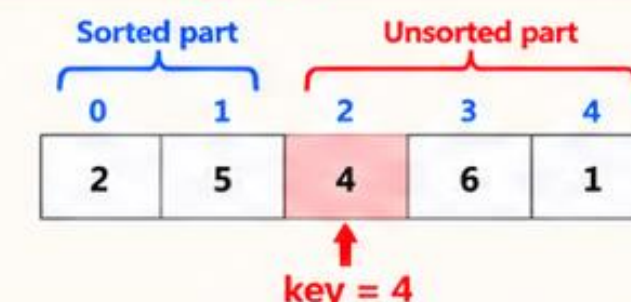
Step 1

Take the element 2 and insert it into the sorted part [5].



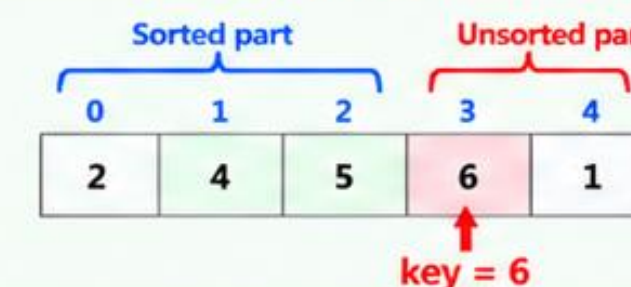
Step 2

Take the element 4 and insert it into the sorted part [2, 5].



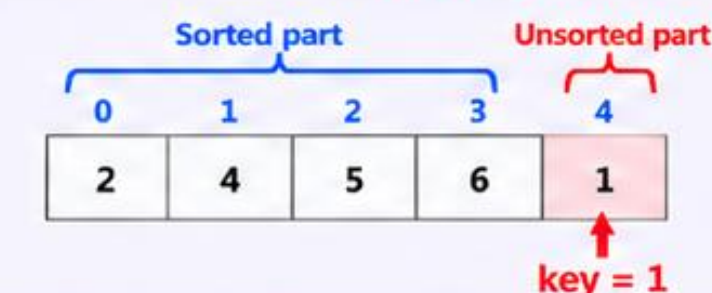
Step 3

Take the element 6 and insert it into the sorted part [2, 4, 5].



Step 4

Take the element 1 and insert it into the sorted part [2, 4, 5, 6].



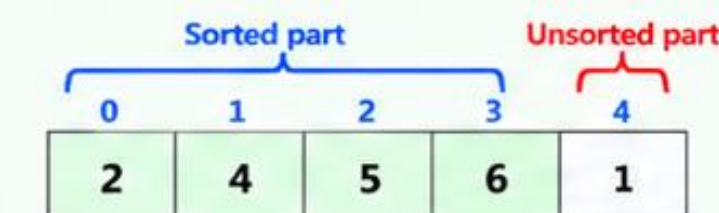
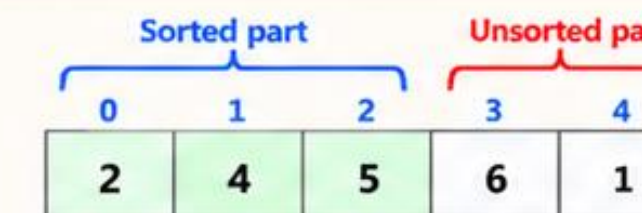
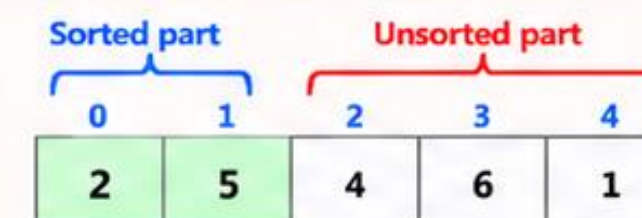
4 Result

Sorted array (ascending order):

0	1	2	3	4
1	2	4	5	6

Initial array:

5	2	4	6	1
---	---	---	---	---



5 Key Points

- Take the next element (*key*).
- Move elements in the sorted part that are greater than the key to the right.
- Insert the key at its correct position.
- Repeat for all elements.
- Time complexity: $O(n^2)$ (simple but easy to understand).

Sorting an Array Using `sort()` in C++

1 What is `sort()`?

- `sort()` is a built-in function in the C++ standard library (in `<algorithm>`).
- It sorts the elements of a range (e.g. an **array** or **vector**).
- By default, it sorts in ascending order (from **smallest** to largest).
- It uses an efficient algorithm (like Introsort).
- Time complexity: $O(n \log n)$.

2 Syntax

```
#include <algorithm>

// Sort the range [first, last)
sort(first, last);
```

- **first** – iterator (pointer) to the beginning of the range
- **last** – iterator (pointer) to one past the last element
- The range `[first, last)` means: from first to last - 1.

3 Example: Sorting an Array

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int arr[] = {5, 3, 8, 1, 4};
    int n = 5;

    sort(arr, arr + n); // ascending order

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

4 Example: Before and After



5 Descending Order

```
sort(arr, arr + n, greater<int>());
```

Result:

0	1	2	3	4
8	5	4	3	1

6 Sorting a vector

```
vector<int> v = {5, 3, 8, 1, 4};
sort(v.begin(), v.end());
```

Result:

0	1	2	3	4
1	3	4	5	8

7 Key Points

- `sort()` is the standard and most commonly used way to sort arrays and vectors in C++.
- It sorts in ascending order by default.
- To sort in descending order, use `greater<int>()`.
- It is much more efficient than simple sorting algorithms (e.g. bubble, selection, insertion sort).
- Time complexity: $O(n \log n)$.

Useful Functions for Arrays in C++

```
#include <algorithm>
```

1 sizeof

Returns the size of the array in bytes.

```
int arr[5] = {1, 2, 3, 4, 5};
cout << sizeof(arr) << "\n";
cout << sizeof(arr[0]) << "\n";
int n = sizeof(arr) / sizeof(arr[0]);
```

If sizeof(int) = 4 bytes:

- sizeof(arr) = 20
- sizeof(arr[0]) = 4
- n = 5 (number of elements)

2 sort()

Sorts the elements in ascending order (default).

```
#include <algorithm>
sort(arr, arr + n);
```

Before:

0	1	2	3	4
5	3	8	1	4



After (ascending):

0	1	2	3	4
1	3	4	5	8

To sort in descending order:

```
sort(arr, arr + n, greater<int>());
```

Result:

0	1	2	3	4
8	5	4	3	1

3 swap()

Swaps two values.

```
swap(arr[i], arr[j]);
```

Before:

0	1	2	3	4
10	20	30	40	50



After:

0	1	2	3	4
10	40	30	20	50

4 reverse()

Reverses the order of elements.

```
reverse(arr, arr + n);
```

Before:

0	1	2	3	4
1	2	3	4	5



After:

0	1	2	3	4
5	4	3	2	1

5 find()

Finds the first occurrence of a value.

```
int* it = find(arr, arr + n, 30);
if (it != arr + n)
    cout << "Found at index " << (it - arr);
else
    cout << "Not found";
```

Array:

0	1	2	3	4
10	20	30	40	50

Output:

Found at index 2

6 count()

Counts how many times a value appears.

```
int cnt = count(arr, arr + n, 2);
```

Array:

0	1	2	3	4	5
1	2	2	3	2	4

Output:

3
(value 2 appears 3 times)

7 min_element()

Returns an iterator/pointer to the minimum element.

```
int* it = min_element(arr, arr + n);
cout << *it; // value
cout << (it - arr); // index (optional)
```

Array:

0	1	2	3	4
7	3	9	1	5

Output:

Value: 1
Index: 3

8 max_element()

Returns an iterator/pointer to the maximum element.

```
int* it = max_element(arr, arr + n);
cout << *it; // value
cout << (it - arr); // index (optional)
```

Array:

0	1	2	3	4
7	3	9	1	5

Output:

Value: 9
Index: 2

9 begin() and end()

Alternative way to specify the range.

```
sort(begin(arr), end(arr)); // same as sort(arr, arr + n)
reverse(begin(arr), end(arr));
find(begin(arr), end(arr), 10);
```

- begin(arr) points to the first element (arr).
- end(arr) points to one past the last element (arr + n).
- Works with arrays and vectors.

- Files are used to permanently store the data on the disk. The data in the main memory is volatile and can not be retained forever.
- Once you close the program, the data of the program is cleared from the main memory so that other programs can use the memory space.
- Therefore files are needed to store the data in the storage devices for a longer time. C++ provides a simple and efficient way to read and write the data into the files. We will now explore the different libraries and their methods to perform the file operations in C++.

- ***ifstream***: You can use this library to read data from a file in C++. For this, you can include the ifstream library in your program to perform file read operations.
- ***ofstream***: You can use this library to write to a file in C++. For this, you can include the ofstream library in your program to write data into a file.
- ***fstream***: You can use this library to perform both the read and write operation on a file in C++. For this, you can include the fstream library to perform all the file operations in your program.

File Processing

Idea

- A file is a file stored on disk that can be used to save and read data.
- We can write data to a file (output) and read data from a file (input).
- This allows us to store information permanently even after the program ends.

Basic steps

- 1 Open a file (for reading or writing).
- 2 Read from or write to the file.
- 3 Close the file.

C++ Example

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // Write to a file
    ofstream fout("numbers.txt");
    fout << 10 << " " << 20 << " " << 30 << endl;
    fout.close();

    // Read from a file
    ifstream fin("numbers.txt");
    int a, b, c;
    fin >> a >> b >> c;
    fin.close();

    cout << "Read values: " << a << " " << b << " " << c;
    return 0;
}
```

1 Writing to a file

```
ofstream fout("data.txt");
fout << 10 << " " << 20 << " " << 30 << endl;
fout.close();
```



data.txt

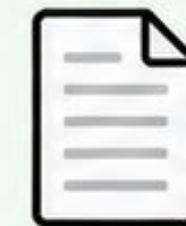


File content (data.txt)

10 20 30

2 Reading from a file

```
ifstream fin("data.txt");
int a, b, c;
fin >> a >> b >> c;
fin.close();
cout << a << " " << b << " " << c;
```



data.txt



File content (data.txt)

10 20 30

Variables after reading:

a = 10 b = 20 c = 30

3 Example: save and read a list of numbers

Write numbers to a file

```
ofstream fout("nums.txt");
int n = 5;
fout << n << endl;    // first line: size
int arr[5] = {1, 2, 3, 4, 5};
for (int i = 0; i < n; i++)
    fout << arr[i] << " ";
fout.close();
```

Read numbers from a file

```
ifstream fin("nums.txt");
int n;
fin >> n;    // read size
int arr[5];
for (int i = 0; i < n; i++)
    fin >> arr[i];
fin.close();

for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
```

File Processing: *ifstream* (Reading from a File)

Idea

- **ifstream** is used to read data from a file.
- We can read text, numbers, and other data using the **>> operator**.
- We must check if the file was opened successfully.
- Always **close** the file after reading.

Syntax

```
#include <fstream>
ifstream filename("file.txt"); // open for reading
filename >> variable;          // read data
filename.close();               // close the file
```

Notes

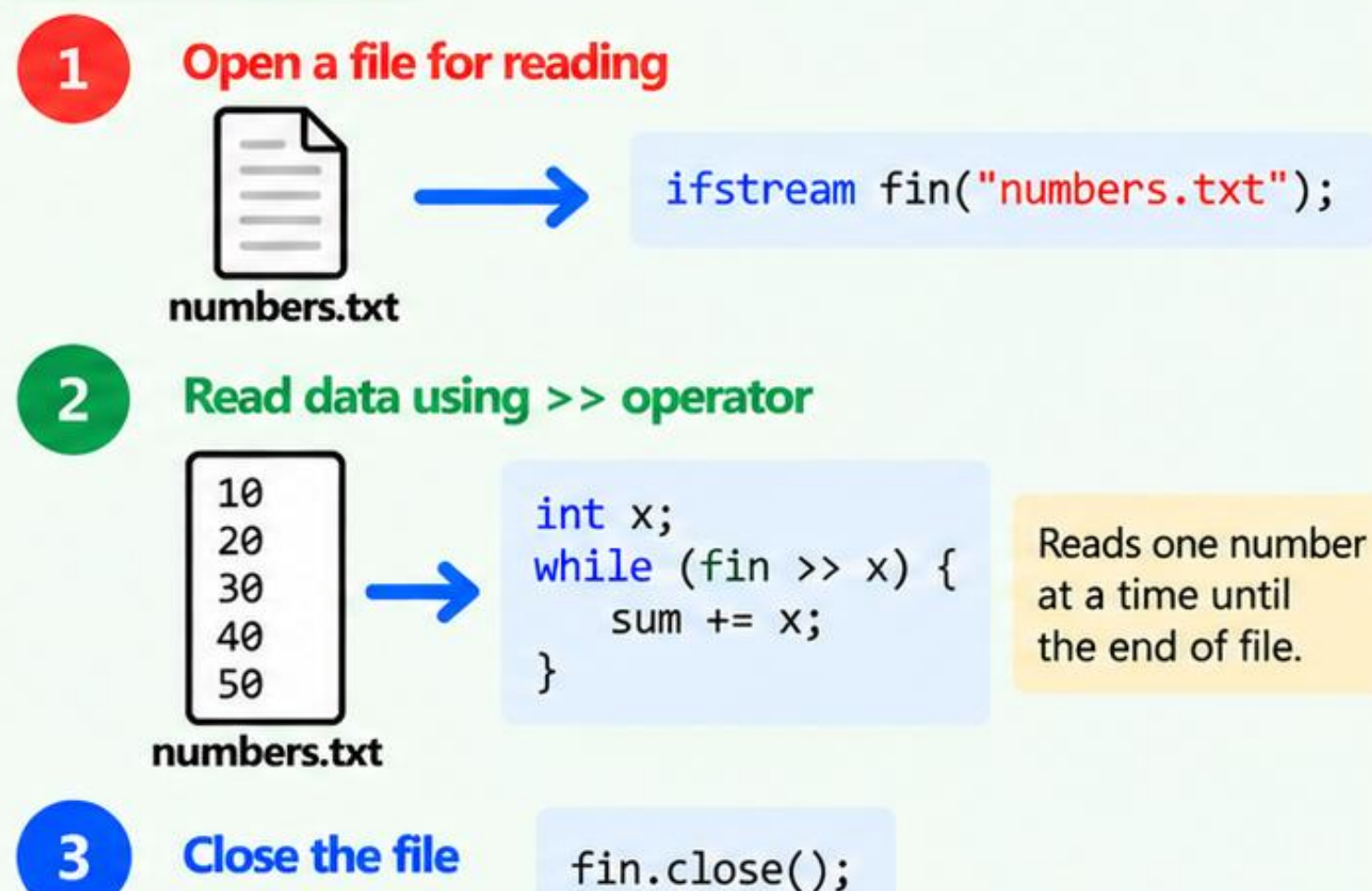
- If the file does not exist or cannot be opened, check with **if (!file)**.
- Data is read using the **>>** operator (works like cin).

Example: Read and process numbers from a file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Open a file for reading
    ifstream fin("numbers.txt");
    if (!fin) {
        cout << "Error: cannot open file!\n";
        return 1;
    }
    // Read numbers and compute their sum
    int x, sum = 0;
    while (fin >> x) { // read until end of file
        sum += x;
    }
    // Close the file
    fin.close();
    cout << "Sum of numbers: " << sum << endl;
    return 0;
}
```

How it works



Input file (numbers.txt)

```
10
20
30
40
50
```

Program output

```
Sum of numbers: 150
```

File Processing: *ofstream* (Writing to a File)

Idea

- **ofstream** is used to write data to a file.
- If the file does not exist, it will be created.
- If the file already exists, its content will be overwritten (unless we use append mode).
- We can write text, numbers, and other data using the **<< operator**.
- Always **close** the file after writing.

Syntax

```
#include <fstream>
ofstream filename("file.txt"); // open for writing
filename << data;                // write data
filename.close();               // close the file
```

Notes

- By default, **ofstream** overwrites the file.
- Use **ios::app** to append to an existing file.
- Always check if the file was opened successfully.

Example: Write student information to a file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // Open a file for writing
    ofstream fout("students.txt");
    if (!fout) {
        cout << "Error: cannot open file!\n";
        return 1;
    }
    // Write data to the file
    fout << "Name \tAge \tGPA\n";
    fout << "Alice\t20\t3.8\n";
    fout << "Bob\t21\t3.5\n";
    fout << "Charlie \t22\t3.9 \n";
    // Close the file
    fout.close();
    cout << "Data has been written to students.txt\n";
    return 0;
}
```

How it works

1

Create and open a file



students.txt



If the file does not exist,
it will be created.
If it exists, its content
will be overwritten.

2

Write data using
<< operator

```
fout << "Name\tAge\tGPA\n";
fout << "Alice\t20\t3.8\n";
fout << "Bob\t21\t3.5\n";
fout << "Charlie\t22\t3.9\n";
```

3

Close the file

```
fout.close();
```

Result (students.txt)

Name	Age	GPA
Alice	20	3.8
Bob	21	3.5
Charlie	22	3.9

File Processing: *fstream* (Reading and Writing)

Idea

- ***fstream*** can be used for both reading and writing in the same file.
- It combines the functionality of ***ifstream*** and ***ofstream***.
- We can open a file in different modes (read, write, or both).
- Useful when we need to read existing data and also modify or append new data.

Syntax

```
#include <fstream>
fstream file(filename, mode);    // open file with mode
file << data;                    // write data
file >> data;                    // read data
file.seekg(position);           // set read position
file.seekp(position);           // set write position
file.close();                   // close the file
```

File Open Modes

Mode	Description
<i>ios::in</i>	Open for reading
<i>ios::out</i>	Open for writing (creates file if not exists)
<i>ios::app</i>	Open for appending (writes at the end)
<i>ios::trunc</i>	Truncate file (clear content)
<i>ios::ate</i>	Open and move to the end
<i>ios::in ios::out</i>	Open for both reading and writing

Example: Read, update and append data using *fstream*

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

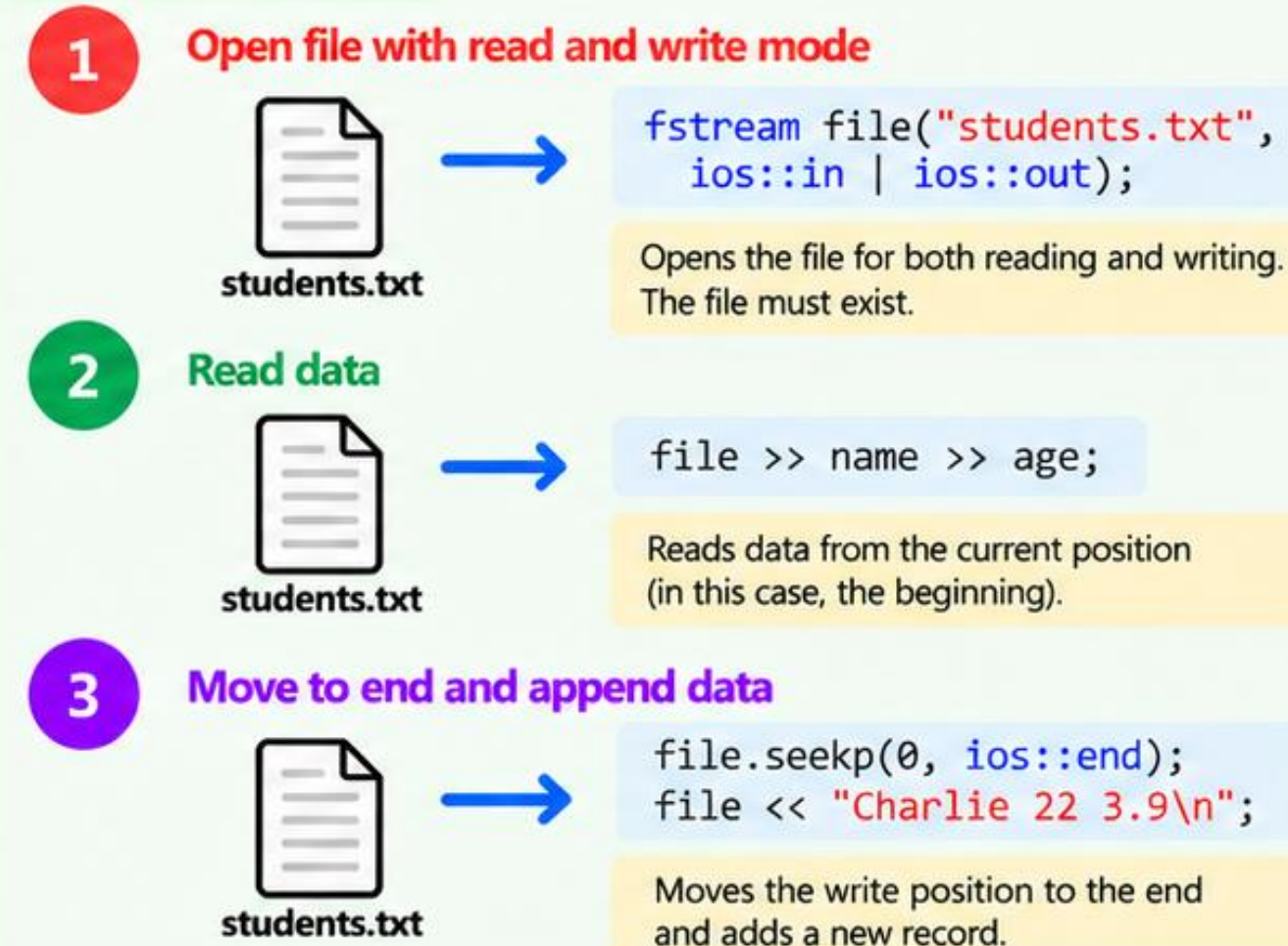
int main() {
    // Open file for reading and writing
    fstream file("students.txt", ios::in | ios::out);
    if (!file) {
        cout << "Error: cannot open file!\n";
        return 1;
    }
    string name;
    int age;

    // Read first record
    file >> name >> age;
    cout << "First student: " << name << " " << age << endl;

    // Move to the end and append a new record
    file.seekp(0, ios::end);
    file << "Charlie 22 3.9\n";

    // Close the file
    file.close();
    return 0;
}
```

How it works



File content before running

students.txt

```
Alice 20 3.8
Bob 21 3.5
```

File content after running

students.txt

```
Alice 20 3.8
Bob 21 3.5
Charlie 22 3.9
```

The freopen() function in C/C++ is used to **redirect input/output** to or from a file instead of the console.

```
freopen (const char * filename, const char * mode, FILE * stream);
```

```
freopen("input.txt", "r", stdin);    // Read input from file  
freopen("output.txt", "w", stdout);  // Write output to file
```

// example of code

```
freopen ("myfile.txt", "w", stdout);  
printf ("This sentence is redirected to a file.");  
fclose (stdout);
```

freopen()

Redirecting Standard Input and Output

Idea

- `freopen()` is a C/C++ function that **redirects a standard stream** (stdin, stdout, stderr) to a file.
- After calling `freopen()`, the standard streams (cin, cout, cerr) work with the file instead of the keyboard or screen.
- It is useful in competitive programming and when testing with files.

Syntax

```
FILE* freopen(const char* filename,
             const char* mode, FILE* stream);
```

- **filename** – name of the file.
- **mode** – how to open the file ("r", "w", "a", etc.).
- **stream** – which standard stream to redirect (stdin, stdout, or stderr).
- Returns a pointer to the opened file, or NULL if failed.

File open modes

Mode	Description
"r"	Open for reading (file must exist)
"w"	Open for writing (create if not exist, truncate)
"a"	Open for appending (create if not exist)
"r+"	Open for reading and writing (file must exist)
"w+"	Open for reading and writing (truncate)
"a+"	Open for reading and appending

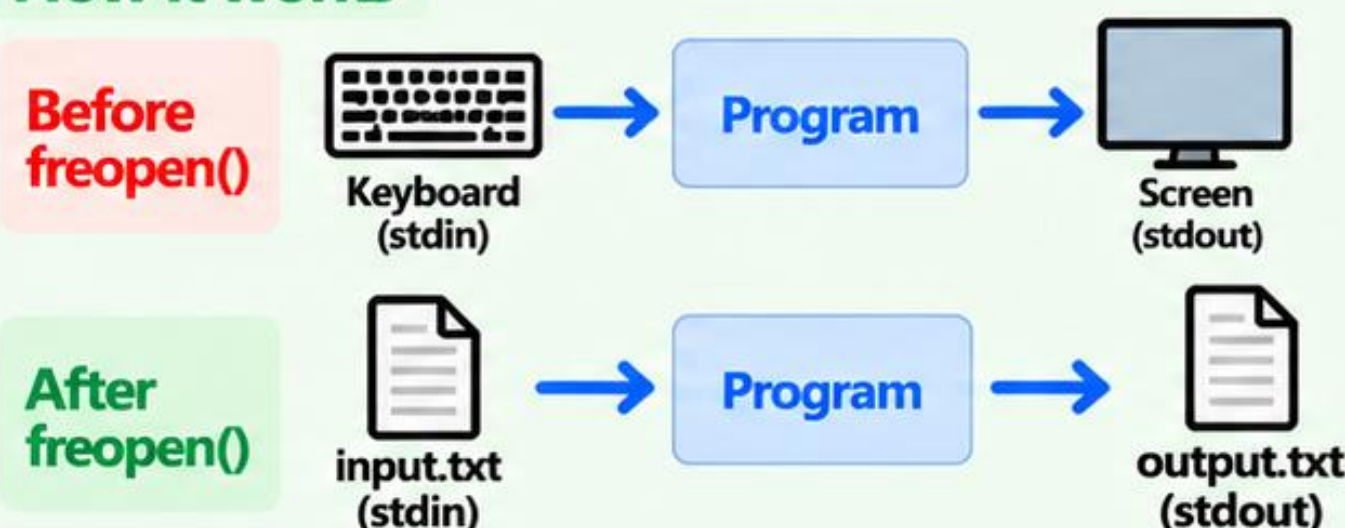
Example: Redirect input and output

```
#include <iostream>
using namespace std;

int main() {
    // Redirect standard input from input.txt
    freopen("input.txt", "r", stdin);
    // Redirect standard output to output.txt
    freopen("output.txt", "w", stdout);

    int a, b;
    cin >> a >> b;
    cout << "Sum: " << a + b << endl;
    return 0;
}
```

How it works



Example: Redirect only output

```
#include <iostream>
using namespace std;
int main() {
    freopen("output.txt", "w", stdout);
    cout << "Hello to file!" << endl;
    return 0;
}
```

Example: Redirect only input

```
#include <iostream>
using namespace std;

int main() {
    freopen("input.txt", "r", stdin);
    int n;
    cin >> n;
    cout << "n = " << n << endl; // to screen
    return 0;
}
```

Important notes

- `freopen()` works with standard C streams (FILE*).
- After redirection, cin and cout automatically use the new files.
- Always check the return value if needed:


```
if (freopen("input.txt", "r", stdin) == NULL) {
    cerr << "Cannot open file!" << endl;
    return 1;
}
```

Additional Resources

<https://www.geeksforgeeks.org/cpp/how-to-read-and-write-arrays-to-and-from-files-in-cpp/>
<https://www.geeksforgeeks.org/cpp/initialize-array-with-values-read-from-a-text-file-in-cpp/>



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Thank You For Your Attention

Questions?

Zhiger Bolatov
z.bolatov@kbtu.kz